



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

Khoa Công nghệ thông tin

Kiểm thử phần mềm

Chương 1: Cơ bản về kiểm thử



1.1 Kiểm thử là gì?

1.2 Tại sao kiểm thử lại cần thiết?

1.3 Nguyên lý kiểm thử

1.4 Các hoạt động kiểm thử

1.5 Kỹ năng cần thiết của người kiểm thử

1.1 Kiểm thử là gì?

1.2 Tại sao kiểm thử lại cần thiết?

1.3 Nguyên lý kiểm thử

1.4 Các hoạt động kiểm thử

1.5 Kỹ năng cần thiết của người kiểm thử

1.1 Kiểm thử là gì?



- **Kiểm thử phần mềm** là quá trình **phê chuẩn**(validation) và **xác minh**(verification):
 - Một phần mềm đáp ứng các yêu cầu hướng dẫn thiết kế và phát triển phần mềm (đáp ứng được các yêu cầu đặt ra ban đầu)
 - Phần mềm hoạt động như mong đợi và đáp ứng nhu cầu của các bên liên quan
 - Phần mềm hoạt động nhất quán
 - Có thể phát hiện ra lỗi (bug) trước khi phần mềm được ban hành

- Nếu không kiểm thử phần mềm thì điều gì có thể xảy ra?
- Bạn nghĩ lỗi nào nghiêm trọng hơn: lỗi giao diện hay lỗi logic (VD: tính toán sai số tiền)?
- Làm sao phân biệt giữa "phê chuẩn" và "xác minh"?

1.1 Kiểm thử là gì?



- **Mục đích của kiểm thử phần mềm:**

- Đánh giá chất lượng phần mềm (dựa theo tài liệu đặc tả của đội phát triển).

Chất lượng phần mềm = Đúng chức năng + Hoạt động tốt + Dễ dùng + Bảo mật + Ổn định

- Giảm thiểu rủi ro khi đưa phần mềm ra thị trường. Ví dụ nổi bật: Lỗi phần mềm trong hệ thống định vị máy bay, ATM, hay thương mại điện tử có thể làm ngưng trệ toàn bộ dịch vụ, có thể làm ảnh hưởng đến tài chính hoặc cả tính mạng.

1.1 Kiểm thử là gì?



- Kỹ thuật kiểm thử: kiểm thử động hoặc kiểm thử tĩnh.
- Kiểm thử không chỉ là một hoạt động kỹ thuật. Kiểm thử cũng cần được lập kế hoạch, quản lý, ước lượng, theo dõi và kiểm soát một cách chính xác.
- Một số quan niệm sai lầm phổ biến về kiểm thử:
 - 📁 Chỉ bao gồm việc execute tests
 - 📁 Tập trung hoàn toàn vào việc verify đối tượng test mà không quan tâm đến cảm nhận của khách hàng

Bài tập tình huống



Tình huống mô tả: Bạn được giao nhiệm vụ kiểm thử chức năng **“Mượn sách”** trong hệ thống quản lý thư viện.

- Chức năng này cho phép:
 - Sinh viên đăng nhập vào hệ thống
 - Tìm sách theo tên, tác giả hoặc mã ISBN
 - Kiểm tra tình trạng còn/mượn của sách
 - Nếu sách còn, sinh viên nhấn “Mượn sách”
 - Hệ thống ghi lại thông tin mượn: người mượn, thời gian mượn, ngày hết hạn
 - Hệ thống gửi thông báo nhắc trả sách qua email 2 ngày trước hạn

Yêu cầu bài tập:

Dưới đây là các hoạt động kiểm thử liên quan đến chức năng **“Mượn sách”**. Hãy phân loại từng hoạt động là **Kiểm thử Tĩnh** (Static Testing) hay **Kiểm thử Động** (Dynamic Testing):

Hoạt động kiểm thử	Tĩnh/ Động
1. Đọc và kiểm tra tài liệu đặc tả để phát hiện mâu thuẫn trong quy định mượn sách?	???
2. Review mã nguồn hàm <code>xacNhanMuonSach()</code> để phát hiện lỗi logic?	???
3. Chạy phần mềm, đăng nhập bằng tài khoản không hợp lệ để kiểm tra xử lý lỗi?	???
4. Kiểm tra tính năng gửi email nhắc sinh viên trả sách?	???
5. Phân tích sơ đồ luồng xử lý nghiệp vụ để xác định bước bị thiếu?	???
6. Thực hiện thao tác mượn sách khi sách đã bị mượn hết để kiểm tra cảnh báo?	???

1.1 Kiểm thử là gì?



- Testing và Debugging

 **Testing** có thể kích hoạt failure do các defect (thiếu sót) trong phần mềm (kiểm thử động) hoặc có thể trực tiếp tìm ra các lỗi trong đối tượng kiểm thử (kiểm thử tĩnh).

 **Debugging** liên quan đến việc tìm nguyên nhân gây ra failure (defect), phân tích nguyên nhân và loại bỏ chúng

-  Tái hiện failure

-  Chẩn đoán (tìm ra nguyên nhân gốc rễ)

-  Khắc phục nguyên nhân

So sánh nhanh: Testing & Debugging



Tiêu chí	☐ Testing	☐ Debugging
Mục tiêu	Phát hiện lỗi	Xác định và sửa lỗi
Ai thực hiện	Tester (kiểm thử viên)	Developer (lập trình viên)
Có sửa lỗi không?	☐ Không	☐ Có
Dựa vào	Test case, đầu vào, hành vi chương trình	Log, phân tích mã, debugger
Giai đoạn	Sau khi có build hoặc module hoàn thiện	Sau khi lỗi được phát hiện

- Một tester phát hiện khi nhập điểm giữa kỳ và điểm cuối kỳ, hệ thống tự động tính điểm trung bình sai.
- Ví dụ:
 - Nhập: Giữa kỳ = 8, Cuối kỳ = 6
 - Hệ thống hiển thị: Điểm trung bình = 10 ?

Quy trình phối hợp (đơn giản hóa):

1. Tester phát hiện → viết bug report
2. Gửi cho developer qua hệ thống quản lý lỗi (Jira, GitLab...)
3. Developer tái hiện → xác định lỗi → sửa
4. Tester test lại (re-test) → nếu ổn → đóng lỗi (close bug)

1.1 Kiểm thử là gì?

1.2 Tại sao kiểm thử lại cần thiết?

1.3 Nguyên lý kiểm thử

1.4 Các hoạt động kiểm thử

1.5 Kỹ năng cần thiết của người kiểm thử

1.2 Tại sao kiểm thử lại cần thiết?



- Hầu hết mọi người đều có trải nghiệm với phần mềm không hoạt động như mong đợi.
- Phần mềm không hoạt động chính xác có thể dẫn đến nhiều vấn đề, bao gồm mất tiền, thời gian hoặc danh tiếng kinh doanh và trong trường hợp nghiêm trọng, thậm chí là thương tích hoặc tử vong.



Some buyers surely couldn't believe their eyes.
Or their luck.

Sky News screenshot by Chris Matyszczyk/CNET



1.2 Tại sao kiểm thử phần mềm lại cần thiết?



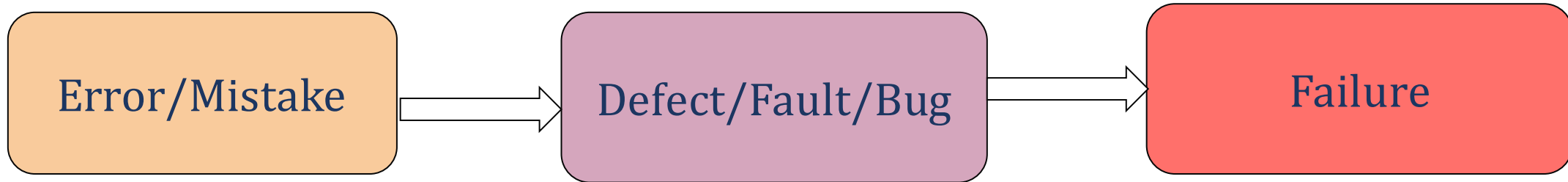
Một số thuật ngữ liên quan đến lỗi:

- Error (mistake): Một hành động của con người tạo ra một kết quả không chính xác
- Defect (fault/bug): Biểu hiện của error trong phần mềm
- Failure: Một sự kiện trong đó một thành phần hoặc hệ thống không thực hiện một chức năng yêu cầu trong các giới hạn được xác định
- Root cause: là nguyên nhân cơ bản dẫn đến sự xuất hiện của vấn đề

1.2 Tại sao kiểm thử phần mềm lại cần thiết?



Một số thuật ngữ liên quan đến lỗi:



*Con người gây ra error, mistake trong code, tài liệu,... => dẫn đến có bug, defect hoặc fault trong code, tài liệu => khi thực thi chương trình thì bắt gặp failure. Và tất cả các thứ này gọi chung là **Issue** hay các vấn đề cần phải giải quyết.*

Ví dụ minh họa: Error, Defect, Failure và Root Cause

- **Hệ thống: Quản lý điểm sinh viên** chức năng chính bao gồm:
 - Giảng viên nhập điểm cho từng sinh viên
 - Hệ thống tính điểm trung bình (GPA) học kỳ và toàn khóa
 - Phân loại học lực: Giỏi, Khá, Trung bình, Yếu
 - Kết xuất bảng điểm PDF cho phòng đào tạo

Ví dụ minh họa: Error, Defect, Failure và Root Cause

1. Error (Mistake) – Sai sót do con người:

- Trong giai đoạn phân tích nghiệp vụ, chuyên viên BA đã mô tả: “Hệ thống xếp loại sinh viên như sau:
 - Giỏi nếu GPA từ 8.0 trở lên
 - Khá nếu từ 6.5 đến dưới 8.0
 - Trung bình nếu từ 5.0 đến dưới 6.5
 - Yếu nếu dưới 5.0”

Tuy nhiên, theo quy chế của Bộ GD-ĐT, mốc xếp loại “Giỏi” bắt đầu từ 8.5 chứ không phải 8.0. → Đây là một Error – lỗi sai từ khâu định nghĩa yêu cầu bởi con người

Ví dụ minh họa: Error, Defect, Failure và Root Cause

2. Defect (Bug) – Lỗi trong phần mềm

Dựa theo yêu cầu bị sai, lập trình viên đã **code logic phân loại** như sau:

```
if (gpa >= 8.0) return "Giỏi";  
  
else if (gpa >= 6.5) return "Khá";  
  
...
```

Với vai trò tester – khi test thử sinh viên có GPA = 8.2 → hệ thống gán loại “**Giỏi**”. -> Đây là **Defect** – phần mềm cho ra kết quả sai theo thực tế

Ví dụ minh họa: Error, Defect, Failure và Root Cause

3. Failure – Hành vi hệ thống không như mong đợi

- Sau khi triển khai cho phòng đào tạo: Một sinh viên có GPA = 8.1 được cấp giấy xác nhận loại Giỏi
- Phòng đào tạo phát hiện sai lệch với quy định, phải thu hồi và sửa lại
 - > Đây là Failure – lỗi gây ra ảnh hưởng thực sự đến nghiệp vụ, làm sai quy trình xét học bổng và tốt nghiệp

Ví dụ minh họa: Error, Defect, Failure và Root Cause

4. Root Cause – Nguyên nhân gốc rễ

Sau khi truy vết, phát hiện:

- BA hiểu sai quy định phân loại học lực của Bộ
 - Không có bản đối chiếu quy định chính thức đính kèm
 - Tester cũng không xác minh với tài liệu từ phòng đào tạo → chỉ kiểm thử theo tài liệu yêu cầu
- > **Root Cause:** Thiếu bước xác minh nghiệp vụ chuẩn, không so sánh với quy định chính thức

- Luôn xác nhận lại các mốc quy định quan trọng với nguồn chính thức (ví dụ: quy chế đào tạo, thông tư của Bộ)
- Áp dụng Test case validation bằng dữ liệu thật (ví dụ từ phòng đào tạo)
- Tester không chỉ test theo tài liệu mà cần có tư duy phản biện

1.2 Tại sao kiểm thử phần mềm lại cần thiết?



=> Kiểm thử tìm bugs hoặc phát hiện failure

=> Đảm bảo chất lượng của sản phẩm

=> Tăng độ tin tưởng và hài lòng của khách hàng

Kiểm thử phần mềm đánh giá chất lượng phần mềm và giúp giảm nguy cơ lỗi phần mềm trong quá trình vận hành.

1.1 Kiểm thử là gì?

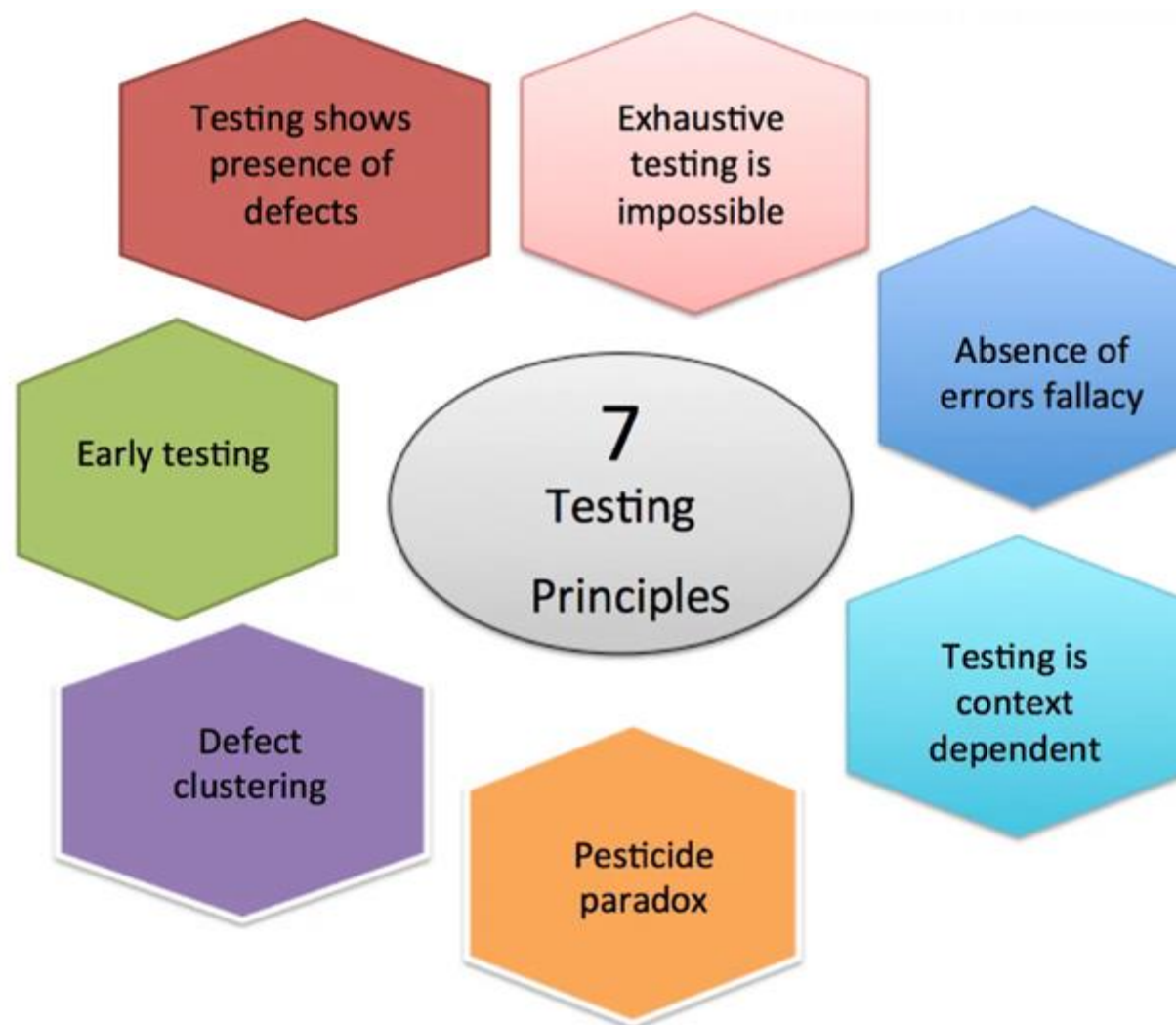
1.2 Tại sao kiểm thử lại cần thiết?

1.3 Nguyên lý kiểm thử

1.4 Các hoạt động kiểm thử

1.5 Kỹ năng cần thiết của người kiểm thử

1.3 Nguyên lý kiểm thử



1. Kiểm thử cho thấy sự hiện diện của lỗi chứ không phải chứng minh phần mềm không có lỗi (Testing shows the presence, not the absence of defects):

- Kiểm thử làm giảm xác suất của các lỗi chưa được phát hiện trong phần mềm
=> Điều này có nghĩa là, sẽ luôn có lỗi không được phát hiện trong phần mềm, ngay cả khi không tìm thấy lỗi, cũng không đồng nghĩa rằng phần mềm đúng hoàn toàn.

- **Tình huống:** Bạn kiểm thử một hệ thống tính điểm sinh viên và chạy **100 test case**, tất cả đều **pass**. -> Bạn **thấy tự tin**, nghĩ rằng phần mềm **không có lỗi**?

Nhưng...

- Test case của bạn **không bao phủ đủ các tình huống đặc biệt**
- Bạn **bỏ sót trường hợp sinh viên bảo lưu, chuyển trường, hoặc học song ngành**
- > Lỗi vẫn tồn tại nhưng **nằm ngoài phạm vi bạn đã kiểm thử**
- Bạn **không phát hiện lỗi**, nhưng **lỗi vẫn tồn tại** → nguyên lý này đúng!

- Đừng bao giờ chủ quan khi thấy test case đều pass
- Tự hỏi: "Còn gì mình **chưa kiểm thử?**", "Còn góc khuất nào chưa được chạm tới?"
- Hãy dùng **phân tích rủi ro, bao phủ test case thông minh**, thay vì cố kiểm thử toàn diện

2. Kiểm thử toàn bộ là không thể (Exhaustive testing is impossible):

- Kiểm thử mọi thứ: tất cả các giá trị đầu vào, điều kiện logic, trường hợp biên, các tổ hợp dữ liệu và hành vi có thể xảy ra
- Thay vì kiểm thử toàn bộ, chúng ta có thể dựa vào đánh giá rủi ro (risks), độ ưu tiên (priorities), sử dụng kỹ thuật thiết kế test phù hợp để kiểm thử.
- VD: Tính điểm tổng kết học kỳ cho sinh viên

Ví dụ: Hệ thống quản lý điểm sinh viên

- Chức năng kiểm thử: Tính điểm tổng kết học kỳ
 - Mỗi sinh viên học 5 môn
 - Mỗi môn gồm các cột điểm: giữa kỳ, cuối kỳ, chuyên cần, bài tập (4 cột)
 - Thang điểm từ 0.0 đến 10.0
 - Trọng số khác nhau giữa các môn (30%-70%, 40%-60%, v.v...)

➔ Bài toán: Bạn muốn kiểm thử hàm “Tính điểm trung bình học kỳ”?

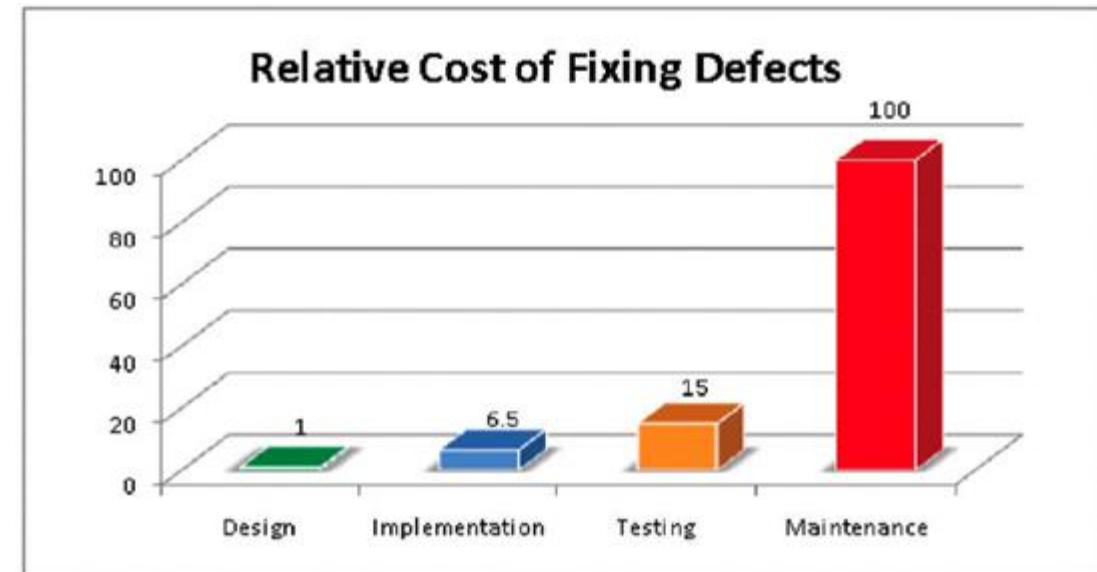
➔ Nếu bạn muốn kiểm tra tất cả các tổ hợp điểm, thì bạn sẽ có:

10 (giá trị điểm giữa kỳ) $\times$ 10 (cuối kỳ) $\times$ 10 (bài tập) $\times$ 10 (chuyên cần) $\times$ 5 (môn học) $\rightarrow = 10^{20}$ tổ hợp $\rightarrow$ không thể nào kiểm thử hết được!

- Là một tester giỏi, bạn **không cần kiểm thử mọi thứ** – bạn cần **kiểm thử đúng thứ cần kiểm.**
- Học cách:
 - Ưu tiên theo mức độ rủi ro
 - Sử dụng kỹ thuật kiểm thử thông minh
 - Phân tích nghiệp vụ để xác định điểm trọng yếu

3. Kiểm thử sớm (Early testing)

- Các lỗi được loại bỏ sớm trong quy trình sẽ không gây ra các lỗi trong các sản phẩm công việc ở các giai đoạn tiếp -> Chi phí về chất lượng sẽ giảm do các phase sau trong SDLC (Software Development Life Cycle) sẽ có ít lỗi xảy ra hơn.
- Để tìm ra lỗi sớm, cả kiểm thử tĩnh và động nên được bắt đầu càng sớm càng tốt.



IBM System Science Institute Relative Cost of Fixing Defects

Ví dụ: Hệ thống quản lý điểm sinh viên

- **Tình huống:** Phân tích yêu cầu sai: BA (Business Analyst) ghi rằng **học phần Giáo dục Quốc phòng không tính vào GPA (Grade Point Average)**, nhưng thực tế lại **phải tính**.
- Nếu lỗi này được phát hiện **ngay trong bước phân tích yêu cầu**:
 - BA sửa ngay → chi phí gần như = 0
- Nếu lỗi bị phát hiện **sau khi đã triển khai xong hệ thống**:
 - Dev phải sửa logic tính GPA
 - Tester phải viết lại test case
 - Phòng đào tạo phải cập nhật dữ liệu
 - Ảnh hưởng đến **tốt nghiệp, học bổng** của sinh viên
 - **Chi phí gấp nhiều lần + ảnh hưởng danh tiếng**

Thực hành kiểm thử sớm gồm những gì?

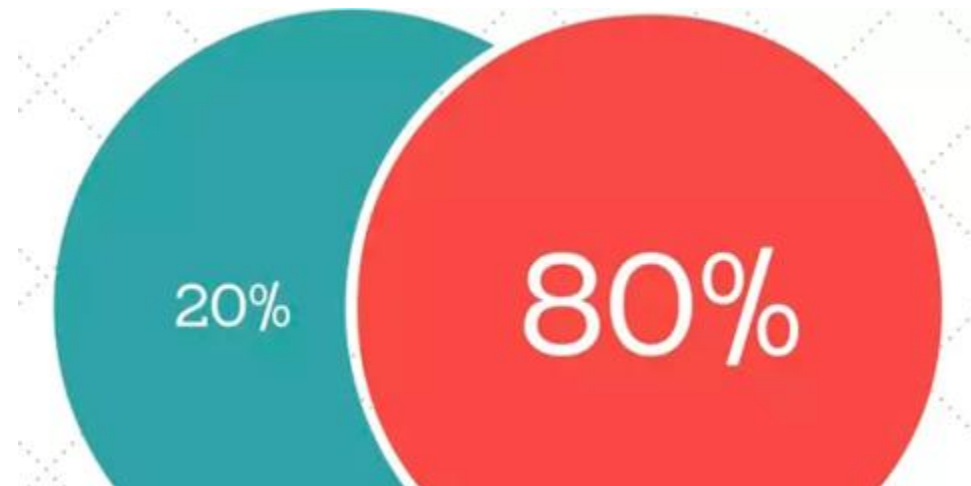


- Review tài liệu sớm: BA, Dev, Tester cùng đọc và kiểm tra tài liệu yêu cầu
- Static testing: Kiểm thử tĩnh – kiểm tra tài liệu, mô hình, không cần chạy phần mềm
- Unit test & Integration test: Được thực hiện ngay từ khi dev viết từng hàm/module
- Automation sớm (Shift-left testing): Đưa kiểm thử vào sớm trong CI/CD pipeline

- Khi tham gia dự án thực tế, đừng nghĩ “chờ phần mềm chạy rồi mới test”.
- Hãy tích cực tham gia từ giai đoạn đầu của dự án, vì chính bạn có thể phát hiện những lỗi chí mạng trong tài liệu, biểu mẫu, thiết kế UI/UX...
- Một tester chủ động ngay từ đầu dự án là một thành viên cực kỳ giá trị!

4. Cụm lỗi (Defects clustering)

- Một số ít các mô-đun của hệ thống chứa hầu hết các lỗi được phát hiện.
- Đây là việc áp dụng Nguyên lý Pareto (80-20 Rule) để kiểm thử phần mềm: khoảng 80% lỗi được tìm thấy trong 20% các mô-đun.



Ví dụ: Hệ thống quản lý điểm sinh viên



- **Các module trong hệ thống:**

- Nhập điểm thủ công
- Import file Excel điểm số
- Tính điểm trung bình học kỳ
- Xét học bổng
- In bảng điểm PDF

- **Quan sát từ thực tế:**

- Lỗi thường xuyên xảy ra nhất ở 2 module:
 - **Tính điểm trung bình học kỳ** (vì có nhiều logic, trọng số, điều kiện đặc biệt)
 - **Import Excel** (vì nhập dữ liệu hàng loạt → lỗi định dạng, thiếu cột, sai dữ liệu...)

-> Nếu dồn 60–70% nỗ lực test vào 2 module này, **hiệu quả phát hiện lỗi sẽ cao hơn nhiều** so với chia đều lực test cho mọi chức năng.

- Đừng chia đều thời gian test cho tất cả mọi chỗ – hãy **tập trung vào những nơi dễ có lỗi nhất.**
- Đọc log bug, thống kê lỗi → học cách **nhận diện “điểm yếu” của hệ thống**
- Áp dụng Pareto trong kiểm thử để làm **ít mà hiệu quả cao**

Nếu trở thành QA Lead, nguyên lý này cực kỳ hữu ích trong **lập kế hoạch test thông minh.**

5. Nghịch lý thuốc trừ sâu (Pesticide paradox)

- Một bộ dữ liệu test được sử dụng lặp đi lặp lại sẽ không tìm được ra các lỗi mới.
- Các bộ dữ liệu test cần phải được xem xét cập nhật thường xuyên để giúp tìm ra lỗi mới.



Ví dụ: Hệ thống quản lý điểm sinh viên



- **Bộ test case cũ:**

- Test điểm giữa kỳ = 5.0
- Test điểm cuối kỳ = 7.0
- Test điểm chuyên cần = 10.0

→ Lặp đi lặp lại: Kết quả lúc nào cũng pass.

- **Nhưng nếu test bộ mới:**

- Test điểm âm (-1)
- Test điểm > 10 (11.5)
- Nhập chữ thay vì số ("abc")
- Để trống cột điểm chuyên cần

→ **Lỗi mới có thể xuất hiện**, như: treo hệ thống, sai công thức, lỗi không hiển thị thông báo...

- Một tester giỏi sẽ không "ngủ quên" trên bộ test case hiện tại sẽ luôn cập nhật, mở rộng và cải tiến bộ test case thường xuyên
- Luôn đặt câu hỏi: “Test case này đã đủ thách thức hệ thống chưa?”
- Thường xuyên tìm cách phá vỡ hệ thống thông minh hơn, giống như hacker có đạo đức!

6. Kiểm thử phụ thuộc vào ngữ cảnh (Testing is context dependent)

Cách tiếp cận, phương pháp, kỹ thuật và các loại kiểm thử tùy thuộc vào ứng dụng.

Ví dụ: Kiểm thử, bất kỳ hệ thống POS tại một cửa hàng bán lẻ sẽ khác với Kiểm thử một máy ATM



- Khi nhận dự án mới, câu hỏi đầu tiên nên là: “Hệ thống này thuộc lĩnh vực nào? Mục tiêu sử dụng là gì?”
- Từ đó, bạn mới xác định được:
 - Dùng kỹ thuật test gì? (Exploratory, Risk-based, Regression, Performance...)
 - Viết test case theo hướng chức năng, nghiệp vụ hay bảo mật?
 - Dữ liệu test nên mô phỏng người dùng phổ thông hay người dùng đặc biệt?

Ví dụ: Hệ thống quản lý điểm sinh viên

Với từng mục tiêu sử dụng khác nhau, cách kiểm thử sẽ rất khác nhau.

1. Nhập điểm cuối kỳ (cho giảng viên): Mục tiêu đảm bảo dữ liệu điểm chính xác và hợp lệ -> Kiểm thử cần tập trung:
 - Logic tính điểm đúng theo trọng số
 - Kiểm thử biên (Boundary): điểm = 0, điểm = 10, điểm âm, điểm >10
 - Ràng buộc: không cho nhập ký tự chữ, ký hiệu
 - Kiểm thử phân quyền: chỉ giảng viên lớp đó mới được nhập điểm
- > **Ngữ cảnh thiên về kiểm thử dữ liệu, validation, phân quyền**

Ví dụ: Hệ thống quản lý điểm sinh viên



2. Ngưỡng cảnh: Xét tốt nghiệp (cho phòng đào tạo): Mục tiêu tự động đánh giá sinh viên đủ điều kiện tốt nghiệp hay chưa -> Kiểm thử cần tập trung:

- Đủ số tín chỉ theo ngành đào tạo
 - Không có học phần bị điểm F
 - $GPA \geq$ mức quy định
 - Môn điều kiện như Quốc phòng – Giáo dục thể chất có đạt?
- > **Ngưỡng cảnh thiên về kiểm thử logic nghiệp vụ phức tạp, liên ngành**

Ví dụ: Hệ thống quản lý điểm sinh viên



3. Ngữ cảnh: Sinh viên xem bảng điểm cá nhân (trên app): mục tiêu hiển thị bảng điểm đúng, dễ đọc, bảo mật -> Kiểm thử cần tập trung:

- Giao diện di động hiển thị đúng điểm, GPA
 - Không cho sinh viên khác xem bảng điểm người khác
 - Tốc độ tải bảng điểm với nhiều môn học
 - Trường hợp lỗi mạng, ứng dụng phản hồi thế nào?
- > **Ngữ cảnh thiên về UI/UX, hiệu năng, bảo mật người dùng**

- Không sao chép test case, chiến lược từ hệ thống này sang hệ thống khác một cách máy móc.
- Rèn luyện khả năng thích nghi và phân tích ngữ cảnh hệ thống
- Khi làm kiểm thử, luôn đặt câu hỏi:
 - “Hệ thống này có gì đặc biệt?”
 - “Điểm rủi ro cao nằm ở đâu?”
 - “Người dùng thực sự sẽ quan tâm điều gì?”

7. Sai lầm về việc không có lỗi (Absence-of-defects fallacy)

- Sản phẩm phần mềm mặc dù không có lỗi vẫn có thể bị khách hàng reject do làm sai yêu cầu của khách hàng.
- Kiểm thử phần mềm không chỉ là việc đi tìm lỗi mà còn phải kiểm tra xem sản phẩm có đạt yêu cầu về nghiệp vụ của khách hàng hay không.
- Ngoài việc verification (do it right) thì validation (do the right thing) cũng nên được thực hiện.
- VD: Sản phẩm không có lỗi nhưng có thể chưa validate

Ví dụ: Hệ thống quản lý điểm sinh viên



- Tình huống:
 - Phần mềm chạy ổn định: nhập điểm, tính GPA, xét học bổng → Không có lỗi
 - Nhưng lại không tính điểm các môn Giáo dục thể chất & Quốc phòng vào GPA
 - Trong khi quy định mới của trường là phải tính
 - Vấn đề:
 - Không phải bug logic → Dev làm đúng theo bản đặc tả
 - Nhưng yêu cầu nghiệp vụ thay đổi mà phần mềm không phản ánh → Khách hàng reject
- ➔ Đây là sai lầm về việc “không có lỗi” nhưng lại “không đúng mục tiêu”

- Luôn hiểu mục tiêu người dùng cuối là gì
- Tham gia sớm vào quá trình phân tích yêu cầu
- Đặt câu hỏi: “Người dùng cần gì ở chức năng này?” thay vì chỉ hỏi “Chức năng này có lỗi không?”

1.1 Kiểm thử là gì?

1.2 Tại sao kiểm thử lại cần thiết?

1.3 Nguyên lý kiểm thử

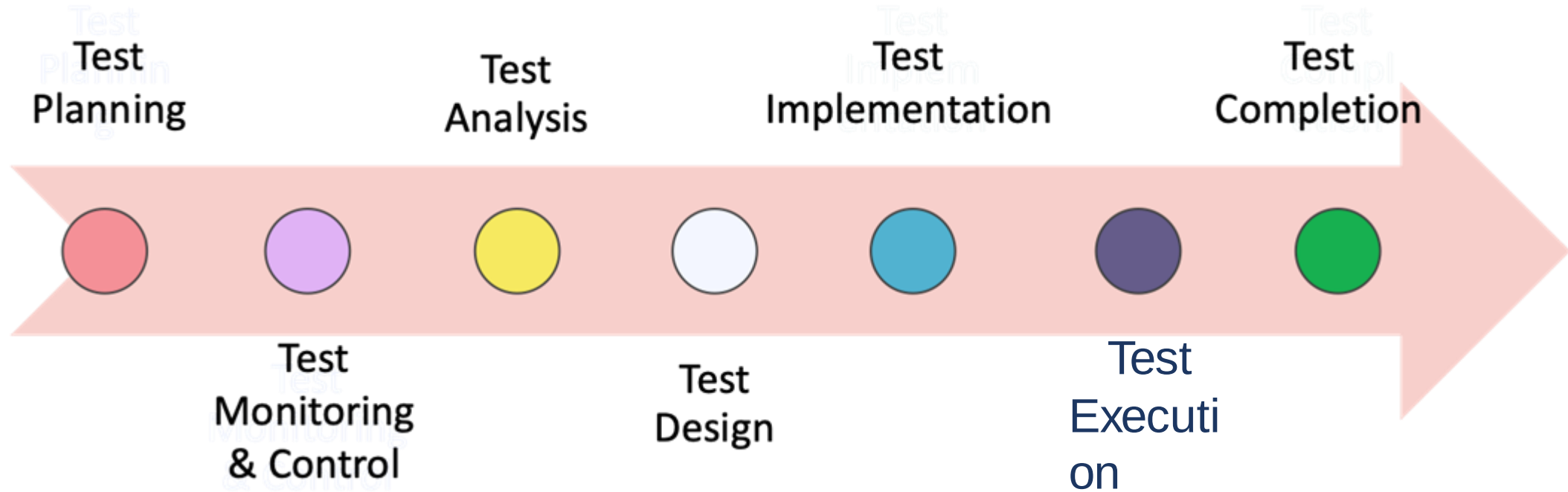
1.4 Các hoạt động kiểm thử

1.5 Kỹ năng cần thiết của người kiểm thử

1.4 Các hoạt động kiểm thử



Một số các hoạt động chính trong kiểm thử phần mềm



1.4.1 Test planning



Kế hoạch kiểm thử (Test planning)

- Xác định phạm vi (scope), rủi ro (risk), và mục đích (objective) việc kiểm thử.
- Phương pháp kiểm thử (test approach).
- Xác định tài nguyên kiểm thử (testing resource) như: Con người, môi trường...
- Lên kế hoạch phân tích kiểm thử, thiết kế kiểm thử, thực hiện kiểm thử và đánh giá.
- Xác định điều kiện kết thúc (exit criteria).



Ví dụ: Hệ thống quản lý điểm sinh viên



Thành phần	Ví dụ cụ thể
Phạm vi	Test phần nhập điểm & xét tốt nghiệp
Rủi ro	Hàm tính điểm có logic phức tạp, dễ sai → cần ưu tiên kiểm thử
Resource	1 QA, 1 Dev hỗ trợ, 1 môi trường UAT (User Acceptance Testing)
Test approach	Kiểm thử chức năng + kiểm thử biên + kiểm thử thủ công
Exit criteria	Toàn bộ test case được thực thi và không có lỗi mức “High” trở lên

- Làm kiểm thử mà không có kế hoạch giống như “ra biển mà không có bản đồ” – dễ mất phương hướng, thiếu phối hợp và khó kiểm soát chất lượng.
- Dù làm đồ án, thực tập hay dự án thật – hãy luôn tập viết Test Plan đơn giản đầu tiên
- Hiểu rõ mình đang test cái gì và tại sao lại test
- Không vội nhảy vào “test case” nếu chưa có kế hoạch kiểm thử tổng thể

1.4.2 Test monitoring and control



Giám sát và Kiểm soát (Test monitoring and control)

- **Test monitoring:** Kiểm tra, so sánh tiến độ thực tế so với kế hoạch, thực hiện báo cáo tình trạng sai lệch so với kế hoạch.
- **Test control:** Đưa ra các hành động cần thiết để giải quyết vấn đề đáp ứng được mục tiêu và nhiệm vụ của dự án.



Ví dụ: Hệ thống quản lý điểm sinh viên



Trong quá trình kiểm thử module tính điểm & xét tốt nghiệp:

- **Monitoring** cho thấy:

- 60% test case đã thực hiện
- 30% trong số đó failed
- 5 lỗi nghiêm trọng chưa fix

→ Đội QA **báo cáo sai lệch với plan ban đầu**: tỷ lệ fail quá cao ở logic tính GPA

- **Control**:

- Giao lại dev check công thức tính điểm
- Điều chỉnh lịch test: dời test phần in bảng điểm sang sau, tập trung test nghiệp vụ điểm

VD: BẢNG THEO DÕI KIỂM THỬ



STT	Module/Test Suite	Tổng test case	Đã chạy	Pass	Fail	Blocked	Tỷ lệ hoàn thành (%)	Ghi chú/Nguyên nhân
1	Nhập điểm	10	10	9	1	0	100%	Fail do sai định dạng dữ liệu
2	Tính điểm trung bình	8	6	4	2	0	75%	Đội dev sửa công thức
3	Xét tốt nghiệp	12	4	3	1	1	33%	Blocked do chưa có dữ liệu test thật
4	Xuất bảng điểm PDF	6	6	6	0	0	100%	OK

Bài học khi làm BTL



- Luôn có 1 người làm “test leader” theo dõi tiến độ
- Ghi log test, bug, tiến độ mỗi ngày vào bảng Google Sheet
- Dựa vào đó để điều phối, không bị trễ tiến độ

Phân tích kiểm thử (Test analysis)

- Phân tích Test basis để xác định những chức năng có thể test được và xác định các điều kiện test liên quan -> tạo ra Test Condition

Note: Test basis là các tài liệu dùng để phân tích kiểm thử (ví dụ: Software Requirements Specification (SRS), User story, Use case, Functional Design Documents,...)

- Phân tích kiểm thử giải đáp câu hỏi "kiểm thử cái gì?"

Ví dụ: Hệ thống quản lý điểm sinh viên



Tên Use Case: Xét tốt nghiệp cho sinh viên

- Mô tả tóm tắt: Khi sinh viên hoàn thành các học phần theo chương trình đào tạo, hệ thống sẽ tự động kiểm tra điều kiện và xác định sinh viên đó có đủ điều kiện tốt nghiệp hay không.
- Test Basis (Cơ sở để phân tích): **yêu cầu nghiệp vụ** thường có trong quy chế đào tạo
 - Đã hoàn thành tất cả học phần bắt buộc và tự chọn
 - Tổng số tín chỉ đạt $\geq$ quy định (ví dụ: 120 tín chỉ)
 - Không còn môn học nào bị điểm F
 - Điểm trung bình tích lũy (GPA) ≥ 2.00
 - Đã hoàn thành các môn điều kiện: Giáo dục Quốc phòng, Thể chất
 - Không vi phạm kỷ luật hoặc nợ học phí

Ví dụ: Hệ thống quản lý điểm sinh viên



■ Phân tích ra Test Conditions

STT	Test Condition (Điều kiện kiểm thử)
1	Sinh viên có đầy đủ tín chỉ (≥ 120) $\rightarrow$ hệ thống cho xét tốt nghiệp
2	Sinh viên thiếu tín chỉ $\rightarrow$ hệ thống thông báo “chưa đủ tín chỉ”
3	Có môn học bị điểm F $\rightarrow$ hệ thống không cho xét
4	GPA < 2.0 $\rightarrow$ hệ thống không cho xét và thông báo lý do
5	Thiếu học phần Giáo dục Quốc phòng hoặc Thể chất $\rightarrow$ hệ thống từ chối
6	Sinh viên đủ điều kiện nhưng vi phạm kỷ luật/nợ học phí $\rightarrow$ hệ thống báo lỗi nghiệp vụ
7	Sinh viên đủ tất cả điều kiện $\rightarrow$ hệ thống sinh quyết định tốt nghiệp PDF
8	Admin có thể tra cứu trạng thái xét tốt nghiệp từng sinh viên

- Cần đọc kỹ Test Basis: có thể là SRS, user story, hoặc quy chế đào tạo
- Nên phối hợp với phòng đào tạo để xác minh logic nghiệp vụ
- Có thể dùng decision table hoặc sơ đồ trạng thái để làm rõ logic phức tạp
- *Test Analysis là quá trình “dịch ngôn ngữ nghiệp vụ thành các điều kiện kiểm thử kỹ thuật”.* Nếu làm tốt giai đoạn này sẽ giúp:
 - Viết test case logic, đầy đủ, không bị sót trường hợp
 - Phát hiện được nhiều lỗi nghiêm trọng ngay cả khi phần mềm không bị crash

Thiết kế test (Test design)

- Bao gồm việc phát triển các Test Condition thành các Test Case và các test-ware khác (các work product tạo ra từ các hoạt động kiểm thử)
- Các hoạt động:
 - Xác định đầu vào của test case
 - Đặc tả yêu cầu dữ liệu kiểm thử
 - Thiết kế môi trường kiểm thử
 - Xác định cơ sở hạ tầng và công cụ cần thiết khác
- Thiết kế kiểm thử giải đáp câu hỏi "Kiểm thử như thế nào?"

Ví dụ: Hệ thống quản lý điểm sinh viên



- Giả sử test condition: “Sinh viên có GPA < 2.0 thì không được tốt nghiệp”
- Test case sẽ bao gồm:
 - Đầu vào: Sinh viên có GPA = 1.95
 - Môi trường test: CSDL staging, bảng điểm đầy đủ
 - Kết quả mong đợi: Hệ thống thông báo “Không đủ điều kiện tốt nghiệp”
 - Thêm vào đó cần chuẩn bị: Tài khoản login sinh viên
 - Dữ liệu bảng điểm

Lưu ý khi viết test case



- Viết test case không phải là sao chép, mà là biến điều kiện kiểm thử thành kịch bản cụ thể, rõ ràng và có thể thực hiện được.
- Đảm bảo test case:
 - Có dữ liệu đầu vào rõ ràng
 - Có kết quả mong đợi cụ thể
 - Dễ hiểu với người khác đọc
 - Luôn chuẩn bị kỹ dữ liệu test & môi trường test trước khi thực thi

Xây dựng test (Test implementation)

- Bao gồm việc tạo hoặc thu thập các test-ware cần thiết để thực hiện kiểm thử (ví dụ: test data, test suite, test scripts,...).
- Tạo manual/auto test-script/test-procedure

Note: test-script là một tập các test-case được sắp xếp và thực hiện theo một trình tự nào đó.

- Tạo test-suites (bộ kiểm thử)

Note: test-suites là tập các test-scripts hoặc test-procedures để thực hiện trong lần chạy test cụ thể.

- Thiết lập và verify môi trường kiểm thử.

Ví dụ: Hệ thống quản lý điểm sinh viên



- Xét tốt nghiệp:
 - Test data: Sinh viên GPA = 1.95, đủ tín chỉ, có điểm F
 - Manual script: Mô tả 5 bước thao tác xét tốt nghiệp từ màn hình người dùng
 - Test suite: “Xét tốt nghiệp” Gồm 6 test case theo các điều kiện xét tốt nghiệp
 - Thiết lập môi trường Import dữ liệu bảng điểm vào hệ thống staging trước khi chạy test

Chú ý khi xây dựng test



- Không bắt đầu test khi mọi thứ chưa được chuẩn bị kỹ.
- Luôn chắc chắn rằng:
 - Có dữ liệu test sạch, rõ ràng
 - Biết rõ chạy test gì trước, test gì sau
 - Đã xác minh môi trường chạy test ổn định

Thực thi test (Test execution)

- Bao gồm running test theo đúng lịch trình thực thi kiểm thử (test runs).
- Thực thi test có thể thực hiện manual hoặc automated.
- Kết quả test thực tế được so sánh với kết quả mong đợi.
- Kết quả test được ghi lại.



Chú ý khi thực thi test



- Test Execution **không phải là "bấm chạy cho xong"**, mà là quá trình:
 - Soi kỹ xem hệ thống phản ứng đúng chưa
 - Quan sát trạng thái bất thường (đơ, chậm, treo...)
 - Ghi lại kết quả chính xác và khách quan

Hoàn thành test (Test completion)

- Những hoạt động này thường xảy ra tại các điểm mốc dự án (ví dụ: release, cuối sprint, hoặc hoàn thành test level) để xử lý những lỗi chưa giải quyết, yêu cầu thay đổi hoặc các mục trong danh sách công việc chưa được thực hiện.
- Chuyển giao các test-ware cho các nhóm phù hợp.
- Môi trường kiểm thử được đóng lại ở một trạng thái được thống nhất.
- Các hoạt động kiểm thử được phân tích để rút ra bài học và cải thiện cho các sprint, lần release tiếp, hoặc dự án tương lai.
- Tạo báo cáo hoàn thành kiểm thử và thông báo cho các bên liên quan.

Ví dụ: Hệ thống quản lý điểm sinh viên



- Cuối sprint “xét tốt nghiệp”:
 - Tổng kết: 25 test case → 22 pass, 3 fail
 - 5 bug: 3 đã fix, 2 pending → ghi rõ lý do chưa fix
 - Báo cáo gửi lại cho phòng đào tạo xác nhận logic
 - Lưu toàn bộ script + test report trên Google Drive team
 - Ghi chú cần cải tiến: “Chưa test đủ luồng sinh viên học song ngành → bổ sung ở sprint sau”

Chú ý khi hoàn thành test



- Làm kiểm thử chuyên nghiệp không chỉ test xong là xong – mà còn phải close test **có trách nhiệm**.
 - Luôn ghi nhận & báo cáo rõ ràng cuối mỗi lần test
 - Để lại tài liệu test tốt → người sau có thể tiếp tục
 - Biết nhìn lại để trở thành tester tốt hơn mỗi sprint

1.4 Các hoạt động kiểm thử

- Các sản phẩm công việc được tạo ra từ các hoạt động kiểm thử

Test planning	test plan, test schedule, risk register, and entry and exit criteria.
Test monitoring and control	test progress reports
Test analysis	test conditions
Test design	test cases, test charters, coverage items, test data requirements and test environment requirements
Test implementation	test procedures, automated test scripts, test suites, test data, test execution schedule, and test environment elements
Test execution	test logs, defect reports
Test completion	test completion report, action items for improvement of subsequent projects or iterations, documented lessons learned

1.5 Vai trò của người tham gia kiểm thử

Vai trò	Nhiệm vụ
Người quản lý (Manager)	Chịu trách nhiệm chung về quy trình kiểm thử, nhóm kiểm thử và dẫn dắt hoạt động kiểm thử Tập trung vào các hoạt động lập kế hoạch kiểm thử, giám sát, kiểm soát và hoàn thành kiểm thử.
Người thực hiện (Tester)	Chịu trách nhiệm chung về khía cạnh kỹ thuật (technical) của việc kiểm thử Tập trung vào các hoạt động phân tích kiểm thử, thiết kế kiểm thử, triển khai kiểm thử và thực hiện kiểm thử



Vai trò của người tham gia kiểm thử hệ thống quản lý điểm sinh viên



Giai đoạn	Việc của Manager	Việc của Tester
Lên kế hoạch	Lập Test Plan, chia deadline	Học tài liệu quy chế đào tạo, xác định test condition
Thiết kế	Phân công người viết test cho từng module	Viết test case cho chức năng xét tốt nghiệp
Thực thi kiểm thử	Theo dõi tỷ lệ Pass/Fail, update dashboard	Chạy test case, log bug: “hệ thống tính GPA sai”
Hoàn tất & báo cáo	Gửi báo cáo kết quả test cho PO	Viết test summary, đề xuất bổ sung kiểm tra phân ngành

1.6 Kỹ năng cần thiết của người kiểm thử



- Kiến thức kiểm thử ví dụ kỹ thuật kiểm thử, phương pháp kiểm thử,... (để tăng hiệu quả kiểm thử)
- Tỉ mỉ, cẩn thận, tò mò, chú ý đến chi tiết, có phương pháp (để xác định các lỗi, đặc biệt là những lỗi khó tìm thấy)
- Kỹ năng giao tiếp tốt, lắng nghe tích cực, làm việc theo nhóm (tương tác hiệu quả với tất cả các bên liên quan, truyền đạt thông tin cho người khác dễ hiểu, biết báo cáo và thảo luận về các lỗi)
- Tư duy phân tích, tư duy phản biện, sáng tạo (để tăng hiệu quả kiểm thử)
- Kiến thức về miền (để có thể hiểu và giao tiếp với người dùng cuối/đại diện người dùng)



1.6 Quiz



TÊN HỌC PHẦN

Bài học tiếp theo:

Tiêu đề bài học tiếp theo

Tài liệu tham khảo:

[1]

[2]

Tư liệu:

[1]

[2]

